

Tuesday, March 10

Midterm 1

Minimum: 46

Median: 74

Mean: 74

Maximum: 99

Standard deviation: 16

Grades in canvas!

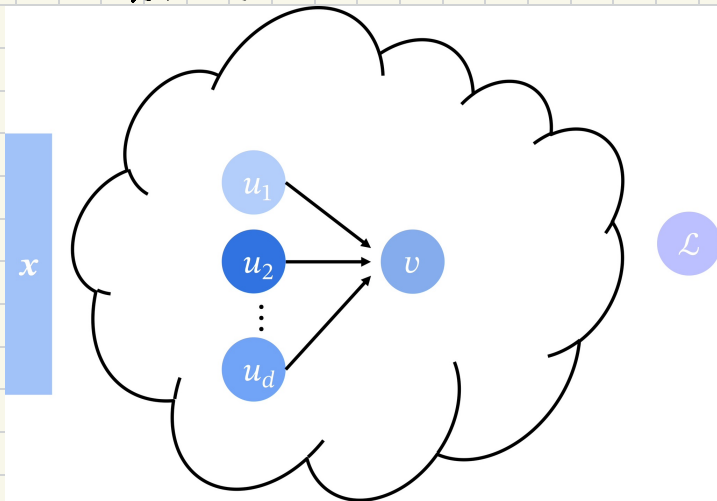
Long tail, will curve eg. 50% to D+/C-

Plan

- Backpropagation
- Convolutional Neural Networks

Forward

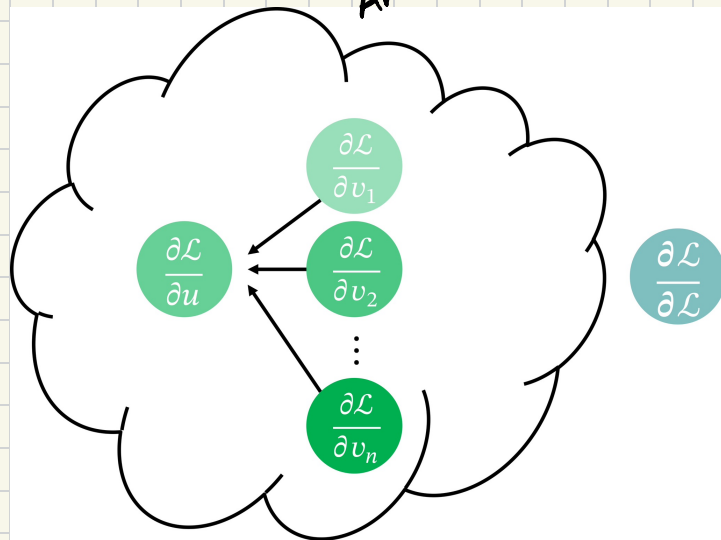
Already know $u_1, \dots, u_d$



$$v(u_1, \dots, u_d)$$

Backward

Already know $\frac{\partial \mathcal{L}}{\partial v_1}, \dots, \frac{\partial \mathcal{L}}{\partial v_n}$



$$\frac{\partial \mathcal{L}}{\partial u} = \frac{\partial \mathcal{L}}{\partial v_1} \frac{\partial v_1}{\partial u} + \dots + \frac{\partial \mathcal{L}}{\partial v_n} \frac{\partial v_n}{\partial u}$$

Linear Algebraic View of Backpropagation

Neural nets known since 1950's

Backprop known since 1960's

Why now?

GPUs used starting 2010's

really good at
matrix multiplication,
so training and inference
cost as matrix multiplication

Forward

$$v = Wu$$

$$v_i = \sum_{j=1}^d [W]_{ij} u_j$$

$$\frac{\partial v_i}{\partial u_j} = [W]_{ij}$$

Backward

$$\nabla_u \mathcal{L} = W^T \nabla_v \mathcal{L}$$

$$\frac{\partial \mathcal{L}}{\partial u_j} = \sum_{i=1}^n \frac{\partial \mathcal{L}}{\partial v_i} \frac{\partial v_i}{\partial u_j} = \sum_{i=1}^n [W^T]_{j,i} \frac{\partial \mathcal{L}}{\partial v_i}$$

Note: Actually want gradient of parameters

$$\frac{\partial \mathcal{L}}{\partial [w]_{ij}} = \frac{\partial \mathcal{L}}{\partial v_i} \frac{\partial v_i}{\partial [w]_{ij}} = \frac{\partial \mathcal{L}}{\partial v_i} u_j$$

$$\nabla_w \mathcal{L} = (\nabla_v \mathcal{L}) u^T$$

Neural networks are a powerful tool for learning complex functions, but notoriously difficult to understand because of their many parameters and nonlinearities. The big surprise of deep learning is that, even though their loss landscapes are not convex functions, gradient descent works remarkably well to train neural networks. We don't have good intuition for what happens in high-dimensional space (the loss function depends on the number of parameters, which can be very large), but it *seems* like there's almost always a parameter update that at least slightly improves the loss function so we rarely get stuck in a bad local minimum.

Motivating Convolutional Layers

Goal: Classification on an image

$$\begin{bmatrix} x_{11} & x_{12} & x_{13} \\ x_{21} & x_{22} & x_{23} \\ x_{31} & x_{32} & x_{33} \end{bmatrix}$$

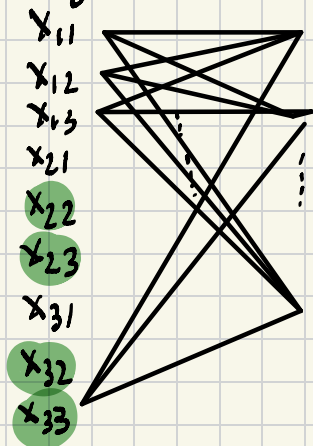
3x3

Each value is color of a pixel

We would like:

1. Fewer parameters in layer
2. Preserve context
3. Shift invariance

Using linear layer...



Q: If 400x400 pixels,
Linear layer where
input has same size as
output, how many parameters?

Convolutional Neural Network

- ✓ locality preserving
- ✓ shift invariant

"kernel"/weights

$$\begin{bmatrix} x_{11} & x_{12} & x_{13} \\ x_{21} & x_{22} & x_{23} \\ x_{31} & x_{32} & x_{33} \end{bmatrix} \star \begin{bmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \end{bmatrix} = \begin{bmatrix} w_{11}x_{11} + w_{12}x_{12} + w_{21}x_{21} + w_{22}x_{22} & w_{11}x_{12} + w_{12}x_{13} + w_{21}x_{22} + w_{22}x_{23} \\ \dots & \dots \end{bmatrix}$$

$$Q: \begin{bmatrix} 1 & 1 & 1 \\ -1 & -1 & -1 \\ 1 & 1 & 1 \end{bmatrix} \star \begin{bmatrix} 1 & 1 \\ -1 & -1 \end{bmatrix}$$

$$\Rightarrow \begin{bmatrix} 1 & -1 \\ 1 & -1 \end{bmatrix}$$

Q: If 400×400 pixels,
convolutional layer with $d \times d$ kernel,
how many parameters?

CNNs continued:

ezyang.github.io/convolution-visualizer/

- Padding: add variables around edges
- Strides: distance between kernel applications
- Pooling: aggregate values
- Transposed Convolution: Increase dimension

$$\begin{array}{c} \text{Input} \\ \begin{bmatrix} 0 & 1 \\ 2 & 3 \end{bmatrix} \end{array} \star^T \begin{array}{c} \text{kernel} \\ \begin{bmatrix} 1 & 1 \\ -1 & -1 \end{bmatrix} \end{array} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix} + \begin{bmatrix} 1 & 1 \\ -1 & -1 \end{bmatrix} + \begin{bmatrix} 2 & 2 \\ -2 & -2 \end{bmatrix} + \begin{bmatrix} 3 & 3 \\ -3 & -3 \end{bmatrix}$$

Q: How do we train?